

Hardware MMU Architecture Specification

32-Bit Translation Pipeline with Frame Management, Two-Tier TLB & OS Handshake

Memory Management Unit Architecture Group

September 15, 2026

Abstract

This document details the architectural specification, finite state machine (FSM) mechanics, and OS-hardware boundaries for a 32-bit hardware Memory Management Unit (MMU). The design combines an address decomposition unit, a 2-tier fully associative TLB cache hierarchy (12-entry L1, 24-entry L2), hardware segmentation and paging table walkers, and an integrated physical frame table tracking process ownership and LRU timestamps. This specification emphasizes state transition graphs, dataflow topologies, and the hardware-software division of labor for fault handling and frame eviction.

Contents

1	System Specifications & Address Partitioning	2
1.1	Virtual Address Decomposition	2
1.2	Structural Module Directory	2
2	System Hardware Topology & Dataflow Architecture	2
3	FSM Controller States & Operational Workflow	2
3.1	Controller State Definitions	3
3.2	FSM State Transition Graph	4
3.3	Signal Transition Matrix	4
4	Physical Frame Management Mechanics	5
4.1	Operational Division in Frame Management	5
4.2	Process Quota Tracking Formulation	5
5	System Interface Boundaries & External Fault Hand-Off	6
5.1	Hardware vs. Software Responsibilities	6
5.2	Segmentation Fault (<code>seg_fault</code>) Flow: Terminal	6
5.3	Page Fault (<code>page_fault</code>) Flow: Recoverable	6
6	External Integration Checklist for Team Members	6

1 System Specifications & Address Partitioning

The MMU coordinates address translations from a 32-bit Virtual Address (VA) into a 26-bit Physical Address (PA) to access a 64 MB physical address space with 1 KB page/frame granularity.

1.1 Virtual Address Decomposition

Incoming addresses are partitioned combinatorially via bit slicing:

$$\text{VA}[31:0] \longrightarrow \underbrace{\text{seg_num}[6:0]}_{\text{Bits [31:25]}} \parallel \underbrace{\text{vpn}[14:0]}_{\text{Bits [24:10]}} \parallel \underbrace{\text{offset}[9:0]}_{\text{Bits [9:0]}} \quad (1)$$

- **Segment Number (seg_num[6:0]):** 7 bits $\rightarrow 2^7 = 128$ segments.
- **Virtual Page Number (vpn[14:0]):** 15 bits $\rightarrow 2^{15} = 32,768$ virtual pages per segment.
- **Page Offset (offset[9:0]):** 10 bits $\rightarrow 2^{10} = 1024$ bytes (1 KB) per page. The offset passes directly into the physical address unmodified.
- **Physical Frame Number (pfn[15:0]):** 16 bits $\rightarrow 2^{16} = 65,536$ physical frames across 64 MB of RAM (26 bits – 10 bits offset = 16 bits PFN).

1.2 Structural Module Directory

Module File	Execution Type			Architectural Responsibility
address_decompose.v	Pure Combinational			Slices 32-bit <code>va_in</code> into <code>seg_num[6:0]</code> , <code>vpn[14:0]</code> , and <code>offset[9:0]</code> .
l1_tlb.v	Hybrid	(Comb / Seq)		12-entry fully associative primary cache. Provides 1-cycle hits; flushed on preemption.
l2_tlb.v	Hybrid	(Comb / Seq)		24-entry fully associative secondary cache. Refills L1 on hit; flushed on preemption.
segment.v	Hybrid	(Comb / Seq)		128-entry table resolving <code>seg_num</code> to Page Table Base Address (<code>pt_base_addr</code>). Flags segment faults.
page_table.v	Hybrid	(Comb / Seq)		Forward translation map ($\text{VPN} \rightarrow \text{PFN}$) resolving <code>pt_base_addr + vpn</code> . Flags page faults.
frame_table.v	Hybrid	(Comb / Seq)		Physical memory ledger ($\text{PFN} \rightarrow \text{PID}$, LRU timestamp, valid status). Tracks process quotas.
mmu_controller.v	6-State FSM			Central orchestrator managing lookup enables, handshakes (<code>busy</code> , <code>pa_ready</code>), TLB refills, LRU touches, and traps.

Table 1: System Module Directory

2 System Hardware Topology & Dataflow Architecture

3 FSM Controller States & Operational Workflow

The translation controller navigates across 6 discrete states.

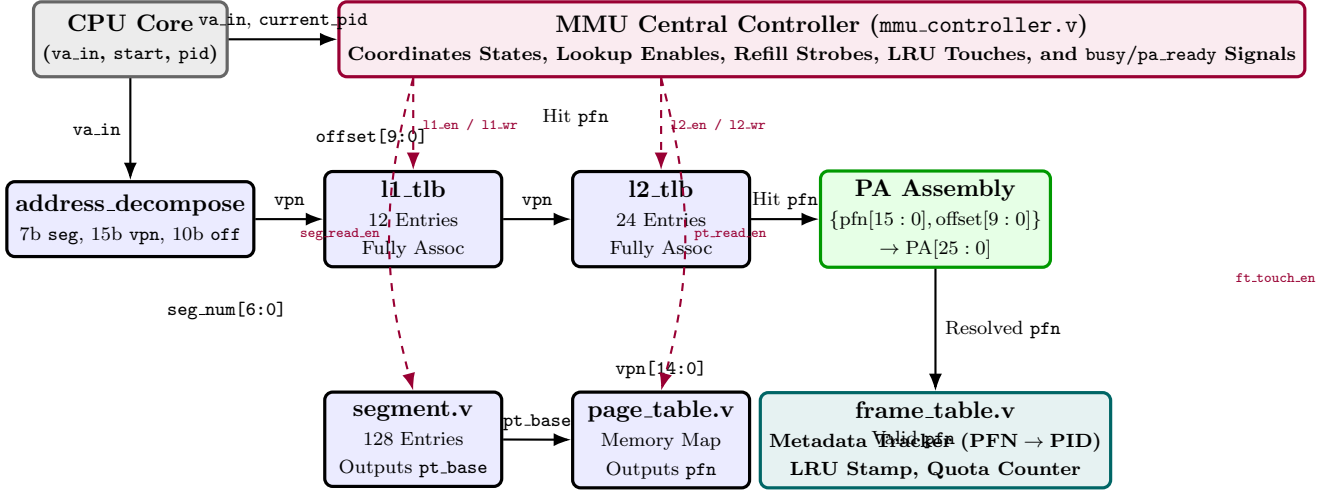


Figure 1: System Hardware Topology: Blue solid lines represent data buses; purple dashed lines represent control strobes.

3.1 Controller State Definitions

1. S_IDLE (3'd0): Quiescent Wait

- Holds the bus quiet. Output signals `busy`, `pa_ready`, `page_fault`, and `seg_fault` remain de-asserted.
- On `start_translate == 1'b1`, samples `va_reg` $\leftarrow$ `va_in`, asserts `busy` $\leftarrow$ `1'b1`, and moves to S_L1_CHECK.

2. S_L1_CHECK (3'd1): Primary TLB Evaluation

- Drives combinational enable `l1_lookup_en = 1'b1`.
- **Hit Path** (`l1_hit == 1`): Sets `pa_out` $\leftarrow$ `{l1_pfn, offset}`, asserts `pa_ready` $\leftarrow$ `1`, drops `busy` $\leftarrow$ `0`, pulses `ft_touch_en` $\leftarrow$ `1` on `l1_pfn`, and transitions directly to S_DONE (1-cycle hit).
- **Miss Path** (`l1_hit == 0`): Advances to S_L2_CHECK.

3. S_L2_CHECK (3'd2): Secondary TLB Evaluation & Backfill

- Drives combinational enable `l2_lookup_en = 1'b1`.
- **Hit Path** (`l2_hit == 1`): Sets `pa_out` $\leftarrow$ `{l2_pfn, offset}`, asserts `pa_ready` $\leftarrow$ `1`, drops `busy` $\leftarrow$ `0`, backfills L1 (`l1_write_en` $\leftarrow$ `1`), pulses `ft_touch_en` $\leftarrow$ `1` on `l2_pfn`, and transitions to S_DONE.
- **Miss Path** (`l2_hit == 0`): Advances to S_SEG_CHECK to initiate a memory table walk.

4. S_SEG_CHECK (3'd3): Segment Descriptor Validation

- Drives combinational enable `seg_read_en = 1'b1` to read `segment.v`.
- **Fault Path** (`seg_fault_wire == 1`): Asserts `seg_fault` $\leftarrow$ `1`, drops `busy` $\leftarrow$ `0`, and jumps to S_DONE.
- **Valid Path** (`seg_valid == 1`): Advances to S_PAGE_CHECK carrying `pt_base_addr`.

5. S_PAGE_CHECK (3'd4): Page Table Walk & Cache Population

- Drives combinational enable `pt_read_en = 1'b1` to read `page_table.v`.
- **Fault Path** (`page_fault_wire == 1`): Asserts `page_fault` $\leftarrow$ `1`, drops `busy` $\leftarrow$ `0`, and jumps to S_DONE.

- **Valid Path** (`page_valid == 1`): Sets `pa_out` $\leftarrow \{\text{page_pfn}, \text{offset}\}$, asserts `pa_ready` $\leftarrow 1$, drops `busy` $\leftarrow 0$, refills both L1 and L2 TLBs (`l1_write_en` $\leftarrow 1$, `l2_write_en` $\leftarrow 1$), pulses `ft_touch_en` $\leftarrow 1$ on `page_pfn`, and transitions to `S_DONE`.

6. `S_DONE` (3'd5): Pipeline Synchronization & Strobe Clear

- Clears all single-cycle active-high strobes: `pa_ready`, `page_fault`, `seg_fault`, TLB write enables, and `ft_touch_en`.
- Returns unconditionally to `S_IDLE` on the next clock edge.

3.2 FSM State Transition Graph

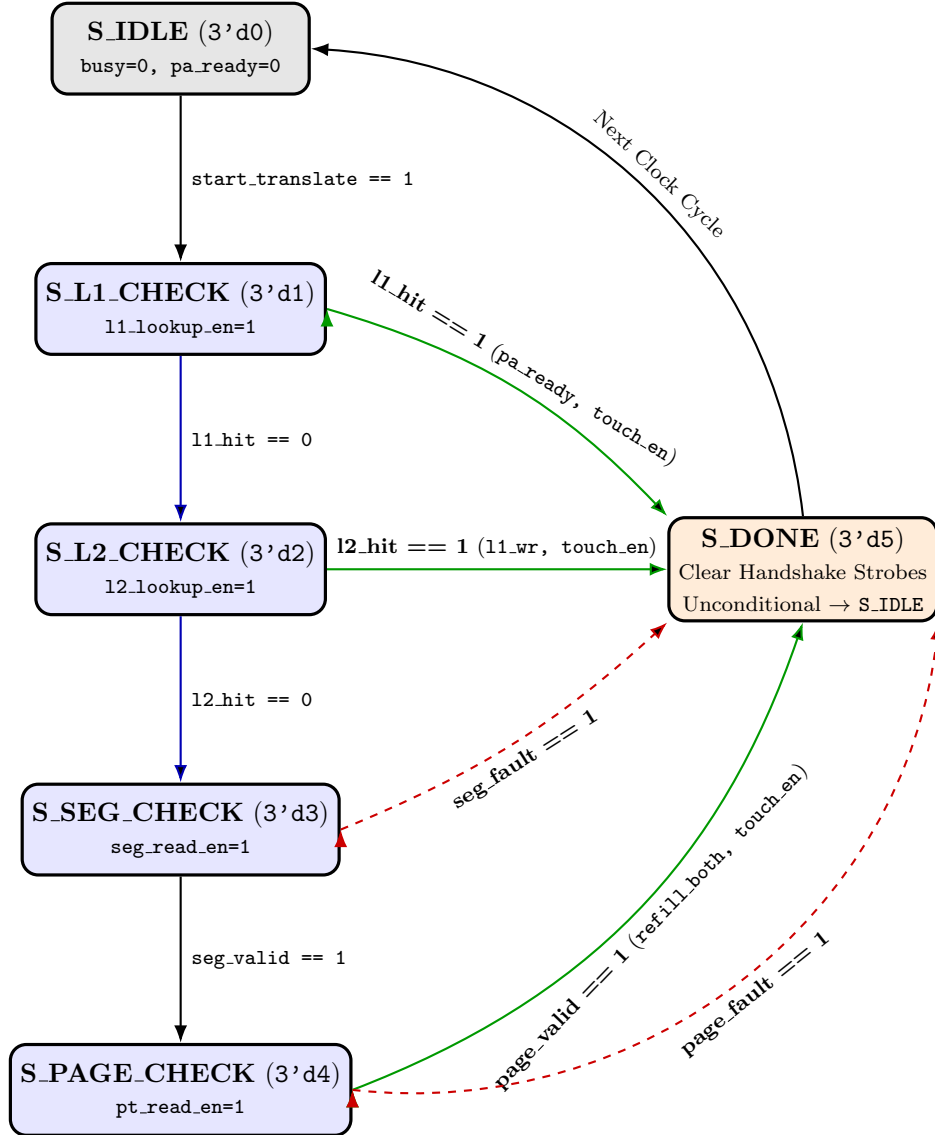


Figure 2: Controller Finite State Machine Graph

3.3 Signal Transition Matrix

Current State	Guard Condition	Next State	Asserted on Edge (≤ 1)	De-asserted on Edge (≤ 0)
S_IDLE	start_translate == 1	S_L1_CHECK	busy, va_reg <= va_in	pa_ready, faults
S_L1_CHECK	l1_hit == 1	S_DONE	pa_ready, ft_touch_en, pa_out	busy
S_L1_CHECK	l1_hit == 0	S_L2_CHECK	None	None
S_L2_CHECK	l2_hit == 1	S_DONE	pa_ready, l1_write_en, ft_touch_en, pa_out	busy
S_L2_CHECK	l2_hit == 0	S_SEG_CHECK	None	None
S_SEG_CHECK	seg_fault_wire == 1	S_DONE	seg_fault	busy
S_SEG_CHECK	seg_valid == 1	S_PAGE_CHECK	None	None
S_PAGE_CHECK	page_fault_wire == 1	S_DONE	page_fault	busy
S_PAGE_CHECK	page_valid == 1	S_DONE	pa_ready, l1_write_en, l2_write_en, ft_touch_en, pa_out	busy
S_DONE	Unconditional	S_IDLE	None	pa_ready, page_fault, seg_fault, write/touch strobes

Table 2: Controller Transition and Signal Assertions

4 Physical Frame Management Mechanics

The hardware FSM coordinates with `frame_table.v` solely to enforce **fast-path LRU timestamping** (`ft_touch_en`), leaving multi-cycle allocation and quota evaluation to the OS maintenance port.

4.1 Operational Division in Frame Management

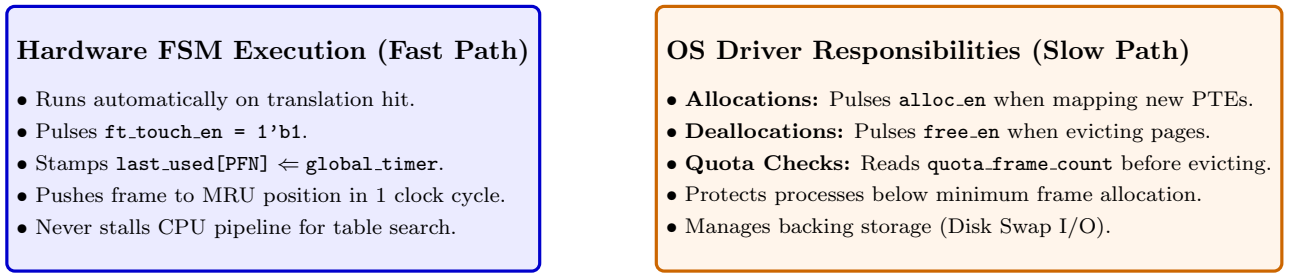


Figure 3: Frame Table Functional Division: Hardware Fast-Path vs. OS Maintenance

4.2 Process Quota Tracking Formulation

To ensure minimum page reservations per process, `frame_table.v` calculates active frame counts combinationally across all physical frames:

$$\text{quota_frame_count} = \sum_{i=0}^{\text{NUM_FRAMES}-1} \left(\text{frame_valid}[i] \wedge (\text{frame_pid}[i] == \text{quota_query_pid}) \right) \quad (2)$$

5 System Interface Boundaries & External Fault Hand-Off

The hardware MMU **only detects and signals faults**. It drops `busy` to prevent deadlock, leaving fault resolution to the Operating System or Testbench.

5.1 Hardware vs. Software Responsibilities

Functional Dimension	Hardware MMU Modules	External OS / Testbench Driver
Address Partitioning	Hardware extraction of <code>seg</code> , <code>vpn</code> , <code>offset</code> .	Allocating virtual memory regions.
Cache Lookups	Parallel comparator matching in L1 and L2.	Supplying <code>preempt_pulse</code> to flush tags on context switch.
Memory Table Walk	Indexes segment and page table entries.	Initializing page table base registers and physical maps.
Frame Metadata Updates	Pulses <code>ft_touch_en</code> on translation hit.	Enforcing eviction policies and handling page allocations.
Segmentation Violations	Detects <code>valid_storage[seg] == 0</code> , asserts <code>seg_fault = 1</code> , drops <code>busy</code> .	Intercepts trap, logs core dump, issues <code>SIGSEGV</code> , terminates process (Terminal).
Page Fault Exceptions	Detects <code>valid_storage[pte] == 0</code> , asserts <code>page_fault = 1</code> , drops <code>busy</code> .	Finds free frame, transfers page from disk, updates PTE via maintenance port, restarts instruction.

Table 3: Hardware vs. Software Responsibility Boundary

5.2 Segmentation Fault (`seg_fault`) Flow: Terminal

- **Root Trigger:** `valid_storage[seg_num] == 1'b0` inside `segment.v`.
- **Hardware Reaction:** Asserts `seg_fault` $\Leftarrow$ 1, drops `busy` $\Leftarrow$ 0, suppresses `pa_ready`, and enters `S_DONE`.
- **External OS Actions Required:**
 1. Capture the `seg_fault` pulse as a non-maskable hardware trap.
 2. Extract offending `va_in` and Program Counter (PC).
 3. Issue a `SIGSEGV` signal and terminate the faulting thread.
 4. Free all frames assigned to `current_pid` using `os_frame_free_en`.
 5. **Never restart or retry the instruction.**

5.3 Page Fault (`page_fault`) Flow: Recoverable

6 External Integration Checklist for Team Members

To complete full-system verification, verify that companion modules and software drivers provide the following:

1. **Sarath's Eviction Driver:**

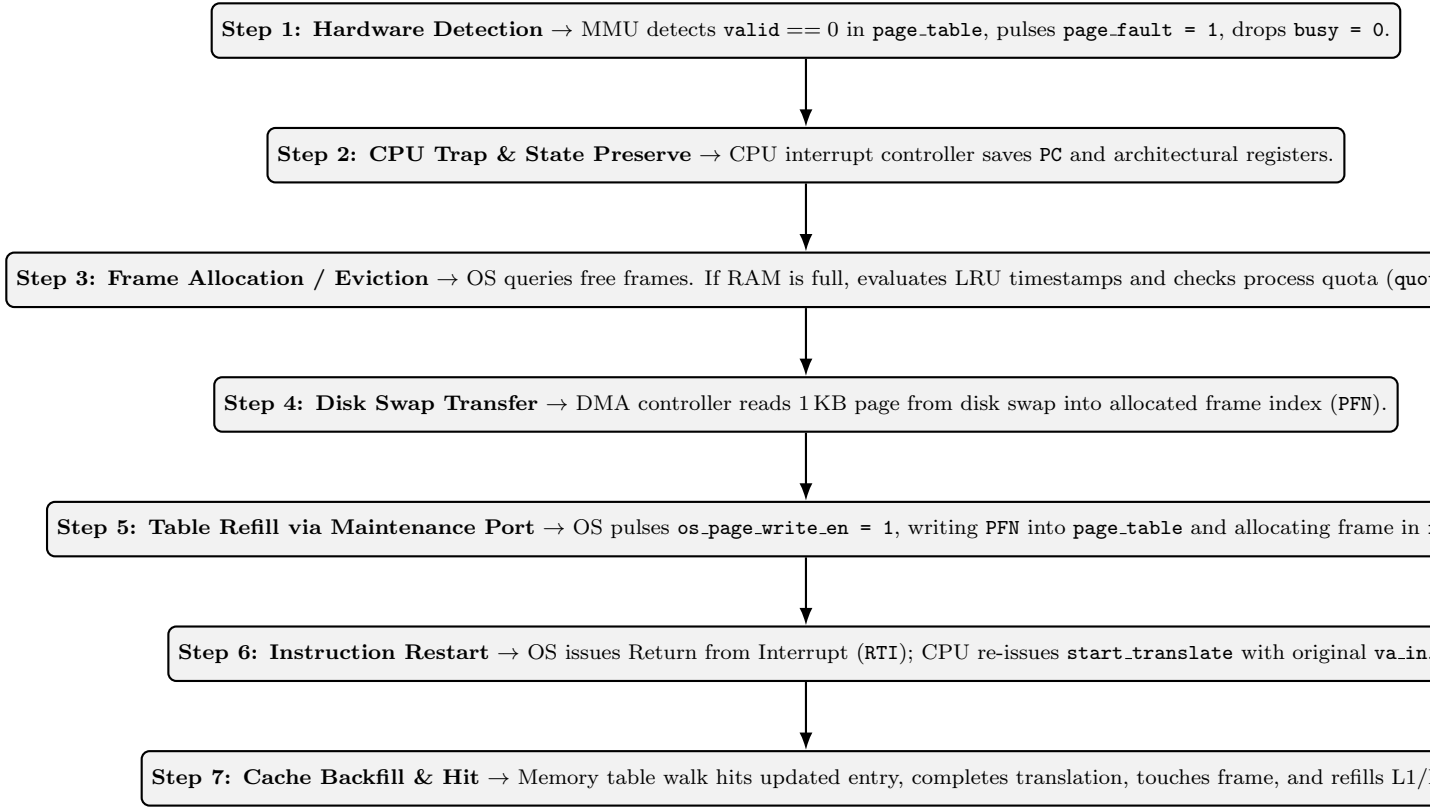


Figure 4: Page Fault Recovery Pipeline (Detection to Instruction Restart)

- Uses `quota_query_pid_toread`
- Guarantees that a process owning $\leq \text{MIN_FRAMES}$ is bypassed during page eviction.
- Asserts `os_frame_free_en` and

2. OS Maintenance Write Driver:

- Directs `os_page_write_*line` to update page mappings following swap-in.
- Maintains identical segment base addresses in `segment.v`.

3. Context Preemption Handler:

- Asserts a 1-cycle pulse on `preempt_p` during every process context switch to flush both TLBs.